

RAP 後端開發練習 8：Associations / Compositions (多層結構)

Lecture

這一課要解決什麼問題，以及為什麼這一課的建立方式跟前面不一樣

rap02 ~ rap07 全部都是「單一實體」的 RAP BO——一張表對一個 CDS View 對一組 Behavior。但真實業務資料幾乎都是**多層結構**：一張訂單 (Header) 底下有好幾筆訂單項目 (Item)，刪除訂單要連帶刪除所有項目，新增訂單常常要「連同項目一次送出」。RAP 用 **Composition (組合)** 表達這種父子關係，用 **Association (關聯)** 讓子實體反向指回父實體。

這一課開始，物件建立方式改變：rap01 ~ rap07 的主要課程物件都是 Claude 用 ADT API 直接建立 ([.claude/rules/sap-adt-mcp.md](#) 記錄的 workaround)，但這是課程一開始就講好的**教學分工原則** (見 README)——這樣的教法有個缺口：如果自始至終都是 Claude 建物件，你不會練到「用 Eclipse 精靈實際建立一組 RAP 物件」這個真實工作最常用的操作。rap05 ~ rap07 因為都是在既有物件上加欄位、加語法，還可以沿用「Claude 建立、你事後核對」的模式；但這一課要建**全新的兩張表 + 兩個 CDS View + 一個 BDEF + 一個實作類別**，份量跟複雜度都達到「應該讓你自己動手建」的門檻——這一課的主要物件，改成由你在 **Eclipse ADT** 建立，**Claude** 負責事後驗證 (讀取比對、語法檢查) 與除錯。

不過在把步驟交給你之前，Claude 已經用暫時性的驗證物件 (**scratch objects**，跟正式課程物件不同名，事後會留在 **\$TMP** 但不算進課程物件清單) 把這一課要教的語法在這個系統上逐字驗證過一輪——包括過程中踩到的兩個跟官方教材不一樣的地方 (下面會詳細說明)，確保接下來給你的步驟跟程式碼是「這個系統上真的能動」的版本，不是照抄官方文件、你自己去試錯。

Part A：Managed Composition 語法 (知識儲備，這系統無法執行，不要求你建立)

在看語法之前先說明一下：這一課的 Part A 跟 rap05 ~ rap07 的 Part A 一樣，純粹是語法知識儲備 (Managed CUD 這系統執行不了，原因見 rap03 Part C)，這裡不要求你在 Eclipse 建立對應物件——你只需要看懂語法、知道跟 Unmanaged 版本的差異即可。

Composition / Association 在 Managed BDEF 的語法元素 (查證官方文件並用 Claude 的驗證物件實測確認)：

- ****composition [0..*] of <子CDS View> as _<別名>****：寫在 CDS Root View 的 **as select from ...** 之後、**{ }** 之前，宣告這個實體「擁有」哪個子實體、關聯數量上限 (**[0..*]** 代表零到多筆)。這是這系統既有標準物件 **C_SalesOrderManage** 本身就在用的語法 (**composition [0..*] of C_SalesOrderItemManage as _Item**)，不是新式 **view entity** 專屬的語法，這系統的舊式 **define view / define root view** 完全支援。
- **association to parent <父CDS View> as _<別名> on \$projection.<子鍵> = _<別名>.<父鍵>**：寫在子實體 CDS View 裡，宣告「回指父實體」的關聯，**to parent** 是固定關鍵字。
- **BDEF 裡 association _<別名> { create; }**：宣告父實體允許透過這個關聯建立子實體 (Create-by-Association，簡稱 CBA)——這是這一課的核心機制：呼叫端可以「建立 Header 的同時，一次把底下的 Item 也建立好」，不用分兩次呼叫。
- **⚠ 子實體的 lock 子句，這系統要用 lock dependent (<子鍵> = <父鍵>)**，不是官方教材常見的 **lock dependent by _<別名>**——Claude 用驗證物件實測：**lock dependent by _Header** 啟用報 "

(" expected, not "by". · 改成 `lock dependent ( order_id = order_id )` 才編譯成功。子實體不用自己管理鎖定，鎖定跟著父實體走，這個子句只是告訴框架「透過哪個欄位對應找到父實體的鎖」。

驗證過的完整 Managed BDEF (Claude 的驗證物件，語法已確認正確，這系統執行不了)：

```
managed;

define behavior for ZI_RAP08_ORDER alias Header
persistent table zrap08_order
lock master
{
  create;
  update;
  delete;

  association _Item { create; }

  field ( mandatory ) description;
  field ( readonly ) created_at, created_by;

  mapping for zrap08_order corresponding;
}

define behavior for ZI_RAP08_ORDER_I alias Item
persistent table zrap08_order_i
lock dependent ( order_id = order_id )
{
  update;
  delete;

  field ( readonly ) order_id, item_id;
  field ( mandatory ) material_desc, quantity;

  association _Header;

  mapping for zrap08_order_i corresponding;
}
```

一個 **BDEF** 檔案可以包含多個 `define behavior for` 區塊——這是延伸「多層結構」的自然結果：Header 跟 Item 的行為定義都寫在同一個 BDEF 物件裡（綁在 Root CDS View 上），不是各自獨立的 BDEF 物件。

Part B：Unmanaged Composition + Create-by-Association——這系統真正能跑的版本

跟 rap05 / rap06 不同（Determination / Validation 在非 Draft 的 Unmanaged 完全沒有宣告式語法可用）、也跟 rap07 一樣（Action 沒有這個限制）——**Composition / Association** 的宣告語法，**Managed /**

Unmanaged 都一樣 (`composition [0..*]` of / association to parent / association _Item { `create;` })，差別只在**實作邏輯**要自己寫。Claude 已經用驗證物件把 Unmanaged 版本從語法到執行完整跑過一輪，過程中踩到一個這系統特有、官方文件沒有明講的關鍵坑（見下方）。

Unmanaged BDEF 語法（沿用 Part A 的規則，`implementation unmanaged in class ... unique;` header）：

```
implementation unmanaged in class zbp_i_rap08_order unique;

define behavior for ZI_RAP08_ORDER alias Header
lock master
{
    create;
    update;
    delete;

    association _Item { create; }

    field ( readonly ) created_at, created_by;
}

define behavior for ZI_RAP08_ORDER_I alias Item
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id;

    association _Header;
}
```

⚠ 這裡有個容易忽略的細節：子實體的 **item_id**（子實體自己的主鍵，使用者建立時要指定）不能標 **field (readonly)**——只有 **order_id**（從父實體帶過來的外鍵）才該是 **readonly**。Claude 一開始兩個都標了 **readonly**，啟用時遇到 **The field "ITEM_ID" of entity "..."** cannot be modified.——因為 Create-by-Association 建立子實體時，**item_id** 是使用者透過 EML 傳進來的資料，不是框架自動決定的值，標成 **readonly** 會直接擋下這次建立。**判斷原則**：**readonly** 該標在「使用者不該手動指定、由框架或你的程式碼決定」的欄位（**order_id** 從父實體帶過來、**created_at/created_by** 由邏輯自動填），使用者需要親自輸入的欄位（**item_id**、**material_desc**）不能是 **readonly**。

Create-by-Association 的 Handler Method：新的 **FOR CREATE <alias>_<association>** 語法

```
METHODS create_item FOR MODIFY
IMPORTING it_create FOR CREATE Header\_Item.
```

- **FOR CREATE Header_Item : _Item** 這個寫法 (反斜線 + 底線開頭的關聯別名) 代表「這是透過 **_Item** 這個關聯觸發的建立」，不是父實體自己的一般 **create**——這是全新的衍生型別，跟 rap03 教過的 **FOR CREATE <alias>** (父實體自己的一般建立) 不一樣。反斜線是 ABAP 語法規則：識別字如果要用地線開頭的名稱 (這裡是關聯別名 **_Item**)，要加反斜線跳脫。

方法本體：

```
METHOD create_item.
  LOOP AT it_create INTO DATA(ls_create).
    LOOP AT ls_create-%target INTO DATA(ls_target).
      INSERT zrap08_order_i FROM @( VALUE #(
        client      = sy-mandt
        order_id     = ls_create-%key-order_id
        item_id      = ls_target-item_id
        material_desc = ls_target-material_desc
        quantity     = ls_target-quantity ) ).
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

- **it_create** 是雙層結構：外層一列代表「對哪一個父實體做 Create-by-Association」，**ls_create-%key-order_id** 是這個父實體的 Key (要用哪個父訂單)；內層 **ls_create-%target** 是一個表格，裝著「這次要為這個父實體建立的所有子實體資料」，每一列對應一筆要新增的 Item。兩層 **LOOP** 是這個 **Handler** 的固定寫法：外層走過所有父實體，內層走過每個父實體要建立的所有子實體。

⚠️ ⚠️ 這系統踩到的關鍵坑：EML 呼叫端要用 **%key**，不是官方範例常見的 **%cid_ref**

官方文件 (**ABENBDL_DETERMINATION_ABEXA**) 的 Create-by-Association 範例，父實體用 **%cid** (暫時代碼) 建立、子實體那段用 **%cid_ref** 反查父實體：

```
" 官方範例寫法 ( 這系統測試失敗，父實體是框架自動編號才需要這樣 )
ENTITY SalesOrder CREATE BY \_SalesOrderItem
  WITH VALUE #( ( %cid_ref = '1' %target = VALUE #( ... ) ) )
```

Claude 一開始照抄這個寫法，語法編譯完全正常、**EML** 呼叫也沒有回報任何錯誤 (**sy-subrc = 0**)，但子實體完全沒有被建立——**FAILED** / **REPORTED** / **MAPPED** 三個回應參數全部是空的，好像什麼事都沒發生。這是這一課排錯過程中最容易誤判的地方：沒有任何錯誤訊息，卻沒有效果。Claude 用一個「哨兵值」技巧 (在 Handler Method 裡先無條件插入一筆固定資料，確認方法本身有沒有被呼叫到) 才排查出：Handler Method 有被呼叫，但傳進來的 **it_create** 是空表——問題出在 EML 呼叫端「父子關聯」沒有正確建立起來，不是 Handler 邏輯錯。

原因分析：**%cid_ref** 存在的意義是「父實體的正式 Key 要等存檔後才知道」(例如 Managed BDEF 搭配 **numbering:managed** 自動編號)，所以子實體建立時只能先用暫時代碼 **%cid_ref** 反查，等真正的 Key 確定後框架才幫你接起來。但這一課的 **order_id** 是使用者自己指定的值，一開始就知道真正的 Key 是什麼，根本不需要透過 **%cid_ref** 反查——改用 **%key-order_id** 直接指定父實體的正式 Key，問題就解決了：

```

MODIFY ENTITIES OF zi_rap08_order
  ENTITY Header
    CREATE FIELDS ( description )
    WITH VALUE #( ( %cid = 'H1' order_id = 'ORD001' description = 'Demo Order' ) )

  ENTITY Header
    CREATE BY \_Item
    FIELDS ( item_id material_desc quantity )
    WITH VALUE #( ( %key-order_id = 'ORD001'
                    %target = VALUE #(
                        ( %cid = 'I1' item_id = '0010' material_desc = 'Widget A'
                        quantity = '5' )
                        ( %cid = 'I2' item_id = '0020' material_desc = 'Widget B'
                        quantity = '3' ) ) ) ) )

  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).

COMMIT ENTITIES.

```

一句話記憶：父實體的 Key 是框架自動編號才需要 `%cid_ref`；父實體的 Key 是使用者自己指定（這一課的情況），直接用 `%key-<欄位>` 就好，不用繞 `%cid_ref` 這一層。這是官方文件沒有明講、只能靠實測才會發現的細節——本質上不是語法錯誤（兩種寫法都編譯得過），是「語意選錯」導致框架靜默地找不到要建立在哪個父實體底下。

✅ 驗證結果（`programrun` 無頭執行，完全成功）

Claude 的驗證物件實測輸出：

```

after commit entities
header found: V001      Verify Order
item rows found: 2
0010 Widget A  5.00
0020 Widget B  3.00

```

一次 EML 呼叫（`ENTITY Header CREATE + ENTITY Header CREATE BY \_Item`），Header 跟兩筆 Item 同時建立成功——這就是「巢狀 CRUD」的效果：呼叫端完全不用先建 Header、拿到 Key 之後再分開建 Item，一次搞定。

Eclipse ADT：建立這一課的物件——Step by Step（輪到你了）

這一課換你動手建立主要物件，跟着下面步驟操作，Claude 已經用驗證物件確認語法完全正確，照抄不會遇到上面記錄過的坑：

1. 建立 Header 表格 `ZRAP08_ORDER`

沿用 rap02 教過的「Eclipse ADT 建立 Transparent Table」流程：對著套件 **\$TMP** 右鍵 → **New** → **Other ABAP Repository Object** → 搜尋 **Database Table** → Name 填 **ZRAP08_ORDER**、Description 填 **RAP08 Order Header Table** → Finish · 貼上以下 DDL：

```
@EndUserText.label : 'RAP08 Order Header Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap08_order {
  key client : mandt not null;
  key order_id : abap.char(10) not null;
  description : text100;
  created_at : timestamp1;
  created_by : syuname;
}
```

存檔 (Ctrl+S) + 啟用 (Ctrl+F3 或工具列 Activate 按鈕)。

2. 建立 Item 表格 **ZRAP08_ORDER_I**

同樣方式 · Name 填 **ZRAP08_ORDER_I**：

```
@EndUserText.label : 'RAP08 Order Item Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap08_order_i {
  key client : mandt not null;
  key order_id : abap.char(10) not null;
  key item_id : abap.char(4) not null;
  material_desc : text100;
  quantity : abap.dec(9,2);
}
```

存檔 + 啟用。

3. 建立 Header CDS View **ZI_RAP08_ORDER**

沿用 rap02 教過的「Eclipse ADT 建立 CDS View」流程：對著 **ZRAP08_ORDER** 表格右鍵 → **New Data Definition** (或 **\$TMP** 右鍵 → New → Data Definition) → Name 填 **ZI_RAP08_ORDER**、Description 填 **RAP08 Order Header View** → 精靈的 Templates 畫面選 **Define View** (不要選帶 **entity** 字樣的版本，這系統不支援，rap02 已經教過這個陷阱) → Finish · 貼上：

```

@AbapCatalog.sqlViewName: 'ZIRAP08ORDER'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP08 Order Header View'
@ObjectModel.compositionRoot: true
@Metadata.allowExtensions: true
define root view ZI_RAP08_ORDER
  as select from zrap08_order

  composition [0..*] of ZI_RAP08_ORDER_I as _Item
{
  key order_id,
  description,
  created_at,
  created_by,

  _Item
}

```

先不要急著啟用——這裡引用了 **ZI_RAP08_ORDER_I**，這個物件還不存在，啟用會失敗，等下一步建好再一起啟用。

4. 建立 Item CDS View **ZI_RAP08_ORDER_I**

同樣方式，Name 填 **ZI_RAP08_ORDER_I**：

```

@AbapCatalog.sqlViewName: 'ZIRAP08ORDERI'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP08 Order Item View'
define view ZI_RAP08_ORDER_I
  as select from zrap08_order_i

  association to parent ZI_RAP08_ORDER as _Header
    on $projection.order_id = _Header.order_id
{
  key order_id,
  key item_id,
  material_desc,
  quantity,

  _Header
}

```

存檔。現在兩個 **CDS View** 互相引用，回到 **ZI_RAP08_ORDER** 一起啟用（Eclipse 通常會自動偵測相依物件，跳出的清單裡勾選兩個一起 Activate；如果沒有自動偵測，兩個都手動各自 Activate 一次，第二次啟用會成功）。

5. 建立 Behavior Definition **ZI_RAP08_ORDER**

沿用 rap03 教過的「Eclipse ADT 建立 Behavior Definition」流程：對著 **ZI_RAP08_ORDER**（Header CDS View）右鍵 → **New Behavior Definition** → Name 固定跟 CDS View 同名（灰的不能改）→ **Implementation Type** 選 **Unmanaged** → Next → Transport（**\$TMP** 直接 Finish）。精靈生成的骨架會有大量註解掉的範例行，整份清空改貼：

```
implementation unmanaged in class zbp_i_rap08_order unique;

define behavior for ZI_RAP08_ORDER alias Header
lock master
{
    create;
    update;
    delete;

    association _Item { create; }

    field ( readonly ) created_at, created_by;
}

define behavior for ZI_RAP08_ORDER_I alias Item
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id;

    association _Header;
}
```

存檔 + 啟用。

6. 用 **Ctrl+1** 生成實作類別骨架

游標點在 header 那一行的類別名稱 **zbp_i_rap08_order** 上 → 按 **Ctrl+1** → 選 **Create behavior implementation class** 並套用 → 選傳輸請求（**\$TMP** 免選）→ Finish。系統會依 BDEF 宣告的操作（**create** / **update** / **delete** / **association _Item { create; }**）自動生成對應的空方法骨架。

7. 補上方法本體邏輯

在生成的骨架裡 (Local Types Include) ，把每個空方法填上邏輯——**Claude** 已經完整驗證過下面這份程式碼，照抄即可：

```
CLASS lhc_header DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS lock FOR LOCK
      IMPORTING it_lock FOR LOCK Header.

    METHODS create FOR MODIFY
      IMPORTING it_create FOR CREATE Header.

    METHODS create_item FOR MODIFY
      IMPORTING it_create FOR CREATE Header\_Item.

    METHODS read_header FOR READ
      IMPORTING it_read FOR READ Header RESULT et_result.
ENDCLASS.

CLASS lhc_header IMPLEMENTATION.

  METHOD lock.
  ENDMETHOD.

  METHOD create.
    LOOP AT it_create INTO DATA(ls_create).
      INSERT zrap08_order FROM @( VALUE #(
        client      = sy-mandt
        order_id    = ls_create-order_id
        description = ls_create-description
        created_at  = cl_abap_tstmp=>utclong2tstmp( utclong_current( ) )
        created_by  = sy-uname ) ).
    ENDLOOP.
  ENDMETHOD.

  METHOD create_item.
    LOOP AT it_create INTO DATA(ls_create).
      LOOP AT ls_create-%target INTO DATA(ls_target).
        INSERT zrap08_order_i FROM @( VALUE #(
          client      = sy-mandt
          order_id    = ls_create-%key-order_id
          item_id     = ls_target-item_id
          material_desc = ls_target-material_desc
          quantity    = ls_target-quantity ) ).
        ENDLOOP.
      ENDLOOP.
    ENDLOOP.
  ENDMETHOD.
```

```

METHOD read_header.
  LOOP AT it_read INTO DATA(ls_key).
    SELECT SINGLE order_id, description, created_at, created_by
      FROM zrap08_order WHERE order_id = @ls_key-order_id INTO @DATA(ls_data).
    IF sy-subrc = 0.
      APPEND VALUE #(
        %key          = ls_key-%key
        order_id      = ls_data-order_id
        description    = ls_data-description
        created_at     = ls_data-created_at
        created_by     = ls_data-created_by ) TO et_result.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

(**Ctrl+1** 生成的骨架可能還包含 **update/delete/Item** 相關的空方法——這些留空即可，啟用時只會出現「operation not implemented」的警告，不影響這一課的驗證目標；如果你想練習補完，可以參考 **create / read_header** 的寫法自己試試看。)

存檔 + 啟用整個類別。

8. 通知 Claude 驗證

完成以上步驟後跟 Claude 說一聲，Claude 會：

1. 讀取比對你建立的物件內容 (**sap_get_source** / GET 讀回)
2. 確認 **sap_inactive_objects** 沒有殘留
3. 建立一支驗證程式 (**ZR_RAP08_DEMO**，這是 **PROG/P**，不受這一課「物件由你建立」的限制，Claude 可以直接建)，用 **programrun** 跑一次 Create-by-Association，確認 Header + Item 真的一次建立成功

如果啟用過程中遇到跟這篇講義描述不一樣的錯誤訊息，把畫面截圖或錯誤文字貼給 Claude，會一起排查 (這系統過去也發生過「講義寫的步驟」跟「Eclipse 實際畫面」有落差的情況，回報後 Claude 會回頭修正講義)。

練習：延伸這個 Header-Item 結構

輪到你了：完成上面 8 個步驟建好基本結構、Claude 也驗證過 Create-by-Association 成功之後，試著自己補完 **update / delete** 的 Handler Method (**FOR MODIFY IMPORTING it_update FOR UPDATE Header / FOR UPDATE Item** 等)，讓這個 Header-Item BO 具備完整的 CRUD 能力。遇到不確定的語法，可以參考這一課 **create / create_item / read_header** 的寫法類推，或直接問 Claude。

Part C：這一課的關鍵發現總結

發現	內容
Composition / Association 宣告語法	Managed / Unmanaged 完全一樣，都支援這系統的舊式 define view (不需要 view entity)
子實體 lock 子句	這系統要 lock dependent (欄位 = 欄位)，不是官方教材的 lock dependent by _別名
Create-by-Association Handler 簽章	FOR CREATE <alias>_<association> ，反斜線跳脫底線開頭的關聯別名
子實體 readonly 欄位	使用者需要輸入的欄位 (如子實體自己的 Key) 不能標 readonly ，只有框架/邏輯決定值的欄位才能標
EML 呼叫端父子關聯寫法	父實體 Key 是使用者自訂值時，用 %key-<欄位> ；只有父實體 Key 是框架自動編號時才需要 %cid_ref ——這系統上這個選錯完全不會報錯，只是靜默地建不出子實體，是這一課最容易踩的坑
這系統能不能真正執行	Managed：❌ (Managed Runtime 白名單限制)；Unmanaged：✅ 已驗證成功

學習目標

- 能寫出 Composition / Association 的 CDS 語法：**composition [0..*] of ... as _<別名>** (父)、**association to parent ... as _<別名> on ...** (子)
- 能寫出這系統適用的多實體 BDEF：一個 BDEF 檔案含多個 **define behavior for** 區塊，子實體用 **lock dependent** (欄位 = 欄位)
- 能寫出 Create-by-Association 的 Handler Method：**FOR MODIFY IMPORTING it_create FOR CREATE <alias>_<association>**，處理雙層 LOOP (外層父實體、內層 **%target** 子實體)
- 知道 EML 呼叫端父實體 Key 是使用者自訂值時要用 **%key-<欄位>** 連結子實體建立，不是官方範例常見的 **%cid_ref** (那是給框架自動編號情境用的)
- 能在 Eclipse ADT 完整建立一組 Header-Item 兩層結構的 RAP 物件：兩張表、兩個 CDS View (Composition + Association to Parent)、一個 BDEF (含兩個 behavior 區塊)、一個實作類別
- 能講出子實體欄位的 **readonly** 判斷原則：使用者要輸入的欄位不能 **readonly**，框架/邏輯自動決定的才能

物件清單

物件	名稱	型別	建立方式	可執行性
Header 表格	ZRAP08_ORDER	TABL/DT	使用者 Eclipse 建立	—
Item 表格	ZRAP08_ORDER_I	TABL/DT	使用者 Eclipse 建立	—
Header CDS View	ZI_RAP08_ORDER	DDLS/DF	使用者 Eclipse 建立	—
Item CDS View	ZI_RAP08_ORDER_I	DDLS/DF	使用者 Eclipse 建立	—

物件	名稱	型別	建立方式	可執行性
Behavior Definition (Unmanaged)	ZI_RAP08_ORDER	BDEF/BDO	使用者 Eclipse 建立	✅ (待使用者建立後 Claude 驗證)
實作類別	ZBP_I_RAP08_ORDER	CLAS/OC	使用者 Eclipse 建立 (Ctrl+1 生成骨架)	✅ (待使用者建立後 Claude 驗證)
EML 驗證程式	ZR_RAP08_DEMO	PROG/P	Claude 建立 (PROG 不受本課分工限制)	待使用者完成建立後，Claude 建立並驗證

Claude 用來驗證語法的暫時性物件 (不算課程正式物件，語法已確認正確，過程記錄在 `.claude/rules/sap-adt-mcp.md`)：ZRAP08V_H / ZRAP08V_I (表格)、ZI_RAP08V_H / ZI_RAP08V_I (Managed CDS + BDEF)、ZI_RAP08VU_H / ZI_RAP08VU_I (Unmanaged CDS + BDEF)、ZBP_I_RAP08VU_H (Unmanaged 實作類別)、ZR_RAP08V_DEMO (驗證程式，已確認 Create-by-Association 端對端成功)。

驗證方式

1. 你依照上方 Eclipse Step by Step 建立 ZRAP08_ORDER / ZRAP08_ORDER_I / ZI_RAP08_ORDER / ZI_RAP08_ORDER_I / ZI_RAP08_ORDER (BDEF) / ZBP_I_RAP08_ORDER
2. 通知 Claude，Claude 讀取比對內容、確認 `sap_inactive_objects` 無殘留
3. Claude 建立 ZR_RAP08_DEMO 並用 `programrun` 執行，驗證 Create-by-Association 一次呼叫同時建立 Header + Item 成功

思考題

1. 這一課的 `create_item` 方法用兩層 LOOP (外層父實體、內層 `%target`)。如果 EML 一次呼叫要對兩個不同的 **Header** (例如 ORD001 / ORD002) 分別建立各自的 Item，這段程式碼不用改就能處理嗎？為什麼？
2. `%key-order_id` (這一課用的，父 Key 是使用者自訂值) 跟 `%cid_ref` (官方範例常見，父 Key 是框架自動編號) ——如果 rap02 的 ZI_RAP02_TASK 改成用 `numbering:managed` 自動產生 `task_id`，Create-by-Association 建立子實體時該用哪一種？
3. Composition 的父子關係也隱含「Cascading Delete」(刪除父實體，子實體一起被刪除) ——這一課的 BDEF 只宣告了 `association_Item { create; }`，沒有另外宣告 `delete` 相關的關聯操作。查證官方文件 ABENBDL_ASSOC_STAND_OPS，Cascading Delete 是不是需要額外宣告，還是父實體 `delete;` 就自動連帶處理？

答案

這一課的主要物件由使用者在 Eclipse 建立，Claude 事後驗證。驗證完成後，答案物件快照 (`zrap08_order.tabl.abap` / `zrap08_order_i.tabl.abap` / `zi_rap08_order.ddls.abap` / `zi_rap08_order_i.ddls.abap` / `zi_rap08_order.bdef.abap` / `zbp_i_rap08_order.clas.abap` / `zbp_i_rap08_order.clas.locals_imp.abap` / `zr_rap08_demo.prog.abap`) 會在你完成建立、Claude 驗證成功後補上。